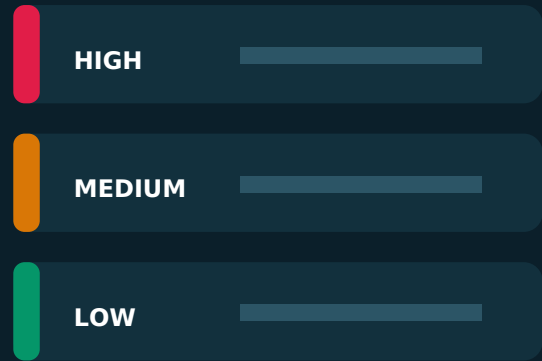


PROJECT REPORT | REVISED EDITION

Support Ticket Priority Prediction & Automated Assignment



Delivered as the SupportIQ ticket intelligence dashboard

React 19

Vite 8

Tailwind 4

Recharts 3

Based on static review of the source; the application was not executed for this report.

Summary

SupportIQ reads a customer ticket, decides whether it is **High, Medium or Low** priority, assigns the least-loaded available agent, and escalates urgent cases with follow-up tasks. A manager can review any low-confidence decision and override it. The dashboard shows SLA risk and agent workload so problems are visible early.

Scope. This report comes from reading the source code. The app was not run, so no screenshots or measured test results are included. Sections that need real data are marked **TBD** or proposed.

What changed in this revision

- **Visual identity.** Teal and sky accent colours replace the previous indigo and purple; the sidebar uses a deep teal-ink surface with a ringed active item.
- **Typography.** Manrope replaces Inter for all interface text.
- **Honest wording.** References to Agentforce and Salesforce Service Cloud were removed, because no such integration exists. The AI is described as the SupportIQ rule engine.
- **No remote photos.** Profile and agent photos loaded from Unsplash are replaced by initials avatars, so the UI works offline and loads faster.
- **Accessibility.** A visible teal keyboard focus ring is applied across the app.

1. Introduction

Support teams receive many tickets each day and prioritise them by hand. This project automates the first steps: classify urgency, route to an agent, and trigger escalation. A support manager can create tickets, see the classification with its rationale and confidence, escalate critical incidents, override doubtful decisions and watch SLA and workload.

Current maturity: front-end prototype

- No backend. Data is stored in browser `localStorage`, seeded with 12 tickets and 6 agents.
- The "AI" is a keyword rule engine, `analyzeTicket()`, with fixed confidence values (62, 86, 93, 96).
- Several dashboard and analytics numbers are hard-coded rather than derived from tickets.

Objectives

- Classify tickets as High, Medium or Low from the description.
- Show the rationale, detected keywords and confidence for every result.
- Assign each ticket to the least-loaded available agent.
- Escalate High tickets automatically and create three urgent tasks.
- Send tickets under 70% confidence to a manager review queue with priority override.
- Monitor SLA risk and agent workload.

2. Ideation

Problem statements

ID	Who	Wants to	But	Because	Feels
PS-1	Support agent	focus on critical tickets first	tickets are not always prioritised automatically	prioritisation is manual	frustrated
PS-2	Customer	get urgent issues fixed fast	critical issues wait behind minor ones	ticket volumes are large	concerned
PS-3	Support manager	route tickets to the right level	assignment needs manual decisions	urgency is judged inconsistently	overwhelmed
PS-4	Support agent	spot urgent issues in text	keywords are missed	descriptions vary	uncertain
PS-5	Support manager	reduce SLA risk	old tickets go unnoticed	SLA risk needs extra checking	concerned

Empathy map (Support agent / manager)

Thinks and feels Wants critical issues seen first; unsure when the AI is not confident.	Hears Issue descriptions, support requests, escalation alerts from the manager.	Sees Status, category, priority badge, confidence, SLA risk, assigned agent, workload bars.
Says and does Reviews, reassigns, escalates, resolves and overrides low-confidence priorities.	Pain Manual prioritisation, missed urgency signals, delays, older tickets going unnoticed.	Gain Automatic priority with rationale, auto-assignment, auto-escalation, less manual effort.

Options considered and final choice

- Manual ticket registers; spreadsheet tracking; a standalone support portal.
- A CRM with an AI add-on (a possible long-term direction, not built).
- **Chosen:** a React single-page dashboard with a service layer and a rule-based classifier.

It was chosen because the team can iterate on triage, escalation and review without a backend, and `ticketService` is the only code that touches data, so mock storage can later be replaced by a real API.

3. Requirements

User journey

Ticket creation > Description analysis > Priority and category > Review > Agent assignment > Urgent escalation > Support handling > Resolution. The first three steps happen on the Create Ticket page; the rest continue on Ticket Details, Escalations and the Dashboard.

Functional requirements

ID	Requirement	Status in code
FR-1	Ticket creation and management (create, list, view, resolve)	Implemented
FR-2	Account and contact association	Implemented (names from mock lists)
FR-3	Ticket description analysis	Implemented (keyword rules)
FR-4	Automatic priority: High / Medium / Low	Implemented
FR-5	High-priority escalation with urgent tasks	Implemented (3 tasks, 2-hour due time)
FR-6	Agent assignment, automatic and manual	Implemented (least-loaded; ignores skills)
FR-7	SLA breach-risk display	Partial (static, no live timer)
FR-8	Manual review and override for low confidence	Implemented (Re-run AI does nothing)
FR-9	Search, filters, CSV export	Implemented (CSV breaks on "#")
FR-10	Conversational AI analysis	Not implemented

Non-functional requirements

Area	Current state
Usability	Met: result panel, rationale, colour-coded badges, toasts, initials avatars, keyboard focus ring
Reliability	Deterministic, but substring matching can produce false HIGH results
Performance	Simulated 150 ms / 300 ms latency; about 2.5 s analysis animation
Security	Not met: no authentication or roles
Maintainability	Good: one service module; no automated tests
Scalability	Limited by localStorage; needs a backend

Technology stack

Layer	Technology
Platform	Browser single-page app (Vite 8, React 19)
Routing	react-router-dom 7
Storage	localStorage keys supportiq_tickets and supportiq_agents, seeded from mockData.js
Logic / AI	ticketService (async) with the analyzeTicket() keyword engine
State	React Context: TicketContext, ToastContext
UI	Tailwind CSS 4, Recharts 3, lucide-react, Manrope typeface
Tooling	Vite, oxlint

4. Design

Problem and solution fit

Problem	Solution	Benefit
Manual prioritisation	analyzeTicket reads the text and sets priority	Urgent cases found faster
Urgent issues missed	High-signal keywords (outage, urgent, 502...)	Critical cases get attention
Manual task creation	Escalation attaches three tasks	Less manual effort
Manual assignment	Least-loaded available agent is assigned	Consistent handling
SLA risk overlooked	Risk level, time remaining, SLA Monitor page	Better awareness
AI may be wrong	Confidence score; under 70% goes to review	A person stays in control

Architecture

Layer	Contents
Users	Support manager, support agent
Presentation	10 pages, 17 shared components, Recharts and Tailwind
State	TicketContext (useTickets), ToastContext
Service	ticketService: CRUD, assign, escalate; analyzeTicket rule engine
Data	localStorage today; REST API and CRM connector are planned only

Provider order matters: ToastProvider wraps TicketProvider, which wraps the router and layout. Every service call is async with an artificial delay (150 ms, 300 ms for analysis).

Visual design system (this revision)

Token	Value	Used for
Ink	#0B1F2A	Sidebar, headings, body text
Teal	#0D9488 / #0F766E	Primary actions, active navigation, focus ring
Sky	#0EA5E9 / #0284C7	AI-related elements and chart accents
Surface	#F3F6F7	Page background
Status colours	Rose, amber, green	High, Medium, Low priority (unchanged)
Typeface	Manrope 400-800	All interface text; JetBrains Mono for code

5. Development

Run the project

```
npm install
npm run dev      # Vite dev server
npm run build    # production build to dist/
npm run lint     # oxlint
```

If node_modules was copied from another operating system, delete it and run `npm install` again.

Classification logic (first match wins)

Step	Condition	Result
1	Description under 35 characters and no keyword	MEDIUM, low SLA risk, 62%, General Inquiry, manual review
2	Any high signal (outage, system down, data loss, security breach, urgent, critical, emergency, asap, 502-504, crash, compromised)	HIGH, high SLA risk, 96%; Billing for payment terms, security wording becomes System Outage
3	Any low signal (question, how to, information, documentation, upgrade, pricing, seats, invoice)	LOW, low SLA risk, 93%
4	Otherwise	MEDIUM, 86%; Account Access for login / sso / auth, else Performance

Workflow after classification

- Suggest the available agent with the lowest workload (fallback: Priya Sharma).
- Apply the SLA target: High 2 h, Medium 8 h, Low 24 h. High tickets start as ESCALATED.
- Escalation attaches three tasks: review and respond, manager outage escalation, incident bridge setup.
- $\text{Workload} = \text{round}(\text{open tickets} / 25 \times 100)$, capped at 100.
- Show a toast, and for High tickets add a notification.

State actions

Action	Effect
<code>createTicket(data)</code>	Saves ticket, updates workload, shows toast, notifies on HIGH
<code>assignTicket(id, agent)</code>	Reassigns and recomputes both agents' workloads
<code>escalateTicket(id, reason)</code>	Status ESCALATED, priority HIGH, tasks attached
<code>overridePriority(id, priority, reason)</code>	Records the override; HIGH also escalates
<code>resolveTicket(id)</code>	Status RESOLVED with a resolved date

Routes

Dashboard (/), Tickets, Create Ticket, Ticket Details, My Tickets, Escalations, SLA Monitor, Analytics, AI Insights and Settings share one layout with sidebar and top bar. Dashboard, Analytics and parts of AI Insights contain hard-coded metrics.

6. Testing

Not yet executed. Expected results below come from reading the code. Run each case and record the actual result before submission.

TC	Input / action	Expected result	Actual
1	"Production outage, system down, urgent" (long text)	HIGH, 96%, System Outage, ESCALATED, toast and notification	TBD
2	"Urgent: checkout payment failing for all customers"	HIGH, 96%, Billing	TBD
3	"Security breach, token compromised"	HIGH, 96%, System Outage	TBD
4	"Not sure what is wrong here." (under 35 chars)	MEDIUM, 62%, General Inquiry, manual review	TBD
5	"How do I upgrade my plan? Need pricing for more seats"	LOW, 93%, Billing	TBD
6	"Login via SSO keeps failing after the auth token expired"	MEDIUM, 86%, Account Access	TBD
7	"Dashboard is slow with lag and frequent timeout"	MEDIUM, 86%, Performance	TBD
8	Override a 62% ticket to HIGH with a reason	HIGH, flagged overridden, review cleared, escalated	TBD
9	Reassign a ticket	Old agent count down, new up, workloads recomputed	TBD
10	Resolve a ticket	RESOLVED, date set, success toast	TBD
11	Search "Priya" in the top bar	Opens /tickets filtered by agent	TBD
12	Settings > Reset	12 tickets and 6 agents restored	TBD
13	Check the revised UI at 375 px width	Sidebar collapses behind menu button; initials avatars show	TBD

Known defects to confirm

TC	Input	Expected (current) behaviour
14	"This is not critical, just a cosmetic question"	Marked HIGH (substring match on "critical")
15	"Order 15024 is delayed"	Marked HIGH (number contains 502)
16	Resolve a ticket, check agent workload	Open count does not drop
17	Resolved HIGH ticket on Escalations	Still listed as active
18	Export CSV with "#" in a subject	File may be truncated
19	Click Re-run AI in review modal	Nothing happens
20	Change a setting and reload	Setting is lost and was never applied

7. Advantages and limits

Advantages

- Automatic prioritisation with visible rationale, keywords and confidence.
- Human-in-the-loop review below 70% confidence.

- Automatic escalation with tasks and notifications; least-loaded assignment with manual reassign.
- Operational visibility: SLA risk, workload, filters, search, CSV export.
- One service module as the data boundary, which eases a move to a real backend.

Limits

- Prototype only: per-browser storage, no backend, no authentication or roles.
- Keyword matching ignores context; confidence values are constants.
- Hard-coded dashboard metrics; settings are neither applied nor saved.
- Static SLA time; assignment ignores skills and can pick the manager.
- No automated tests and no TypeScript.

8. Conclusion

SupportIQ shows how rule-based classification, automatic routing and a review gate can cut manual ticket handling. It is a front-end prototype: before production use it needs truthful metrics, applied settings, a backend, a real classifier and authentication.

9. Future scope

- Derive every KPI and chart from live data; apply and persist settings.
- Put a REST backend behind the existing ticketService method names.
- Replace the rule engine with a real classifier or LLM that keeps the same output contract.
- Whole-word matching; skill- and capacity-aware assignment.
- Live SLA timers, real-time notifications, authentication, roles and an audit log.
- Automated tests, CI and TypeScript.

Proposed API (does not exist in this repository)

Endpoint	Maps to
GET /tickets (search, priority, status, slaRisk, category, agent)	getTickets
POST /tickets/analyze	analyzeTicket
POST /tickets	createTicket
POST /tickets/{id}/assign	assignTicket
POST /tickets/{id}/escalate	escalateTicket
POST /tickets/{id}/priority-override	overridePriority
GET /agents/workload	getAgentWorkload

10. Planning

Agile, sprint-based. The repository holds no sprint records, so this backlog is **proposed** from the roadmap and the defect list. Team, dates and actual points are TBD.

Sprint	Epic	Story	SP	Priority
1	Truthful UI	Every KPI and chart derived from live tickets	8	High
1	Truthful UI	Settings applied to analysis and assignment, and saved	5	High
1	Truthful UI	Resolving lowers workload and leaves active escalations	5	High
1	Truthful UI	Re-run AI actually re-analyses	2	Medium
2	Backend	ticketService calls a REST API	8	High
2	Backend	IDs and workload computed server-side	5	High
2	Quality	Whole-word keyword matching	5	High
2	Export	CSV via Blob so "#" exports correctly	2	Medium
3	AI model	Real classifier / LLM with genuine confidence	8	High
3	Assignment	Skills, capacity; exclude the manager	5	High
3	SLA	Live countdown timers	5	Medium
3	Workflow	Action to move a ticket to IN_PROGRESS	2	Medium
4	Security	Authentication and roles	8	High
4	Notifications	Persistent, real-time, respect toggles	5	Medium
4	Quality	Automated tests and CI	5	High
4	Quality	Start TypeScript migration	2	Low

Planned velocity: 80 points over 4 sprints = **20 points per sprint**. Actual velocity is recorded once sprints run.

Start dates, end dates, points completed and release dates are TBD until sprints run.

Appendix: records

Record	Fields
Ticket	id, subject, description, priority, status, issueCategory, slaRiskLevel, slaTargetHours, slaRemainingMinutes, aiConfidenceScore, aiRationale, keywordSignals, requiresManualReview, priorityOverridden, overriddenBy, overrideReason, assignedAgent, account, contact, ticketSource, resolutionTimeHours, createdAt, escalatedDate, resolvedDate, escalationDetails
Agent	id, name, email, role, availability, openTickets, highPriority, workload, skills[], statusText (avatar photo no longer displayed)
Settings	confidenceThreshold (70), maxOpenTickets (20), assignmentAlgorithm, four notification toggles, userAvailable, escalationTierAgent (defined, not applied)
Notification	id, title, message, time, unread, type

Seed roster: Priya Sharma, Rahul Kumar, Anita Joseph, Arun Kumar, Meena Devi, Kathirvel (Support Manager).